

2026-05-04-cli-agent-fusion-synthesis-design

HelixCode CLI-Agent Fusion — Synthesis Design Spec

Date: 2026-05-04 **Author:** Claude Opus 4.7 (1M context) + user (milos85vasic.2nd@gmail.com)
Status: APPROVED in brainstorming, awaiting user review of written spec **Successor:** to be handed to superpowers:writing-plans for executable plan **Supersedes (partially):** the aspirational diagrams in docs/improvements/01_* and 02_* (those used a fictional module set — HelixML/HelixSDK/HelixDB/etc. — that does not exist as real repositories under HelixDevelopment or vasic-digital)

This spec is the single planning artefact for an integration programme that fuses every power-feature, optimisation, and design innovation from every CLI agent in `helix_agent/cli_agents/` into HelixCode, with anti-bluff guarantees applied at every layer.

1. Goals, non-goals, success criteria

1.1 What we're building

A staged programme — *not* a single feature — that produces, in order:

1. A **clean foundation** (governance, secrets, submodule topology) that no later phase can be silently built on top of broken state.
2. A **ported feature set** from claude-code-source first, then every other CLI agent, each landed with runtime-evidence Challenges proving end-user usability.
3. A **verified test infrastructure** spanning unit / e2e / integration / full-automation / performance / concurrency / benchmarking / security / scanning (SonarQube + Snyk + others) / DDoS / Challenge banks / helix_qa banks — every test type configured to fail loudly when bluffing rather than green-PASS on absence-of-error.
4. **Cascaded documentation** — Constitution / CLAUDE.md / AGENTS.md across the meta-repo and every owned submodule, plus user manuals, website, video curriculum, ADRs, and refreshed architectural diagrams.

1.2 Goals (priority order)

- **G1 — No bluffs.** Every PASS in the system carries positive runtime evidence (Constitution Article XI §11.9, CONST-035). Tests/Challenges that go green on a broken feature are themselves the defect.
- **G2 — Single source of truth.** HelixAgent is the integration substrate; `cli_agents` come from `helix_agent/cli_agents/<name>` only — never from `Example_Projects/`.
- **G3 — Secrets safe.** API keys live in `.env` files mode 0600, never in git, never in logs, never in docs.
- **G4 — No force pushes without explicit user approval.** New constitutional mandate cascaded everywhere.
- **G5 — Real production readiness for every ported feature.** “Done” means a Challenge demonstrates end-user usability, not “tests pass.”

1.3 Non-goals (explicit out-of-scope)

- **N1.** Closed-source agents (claude-code core, codex core, warp, amazon-q, copilot-cli, kiro-cli, shai) — we port what's documented/observable, not what's reverse-engineered from binaries.
- **N2.** Replacing or rewriting HelixAgent itself. We consume it as a submodule; if HelixAgent needs changes, that's a separate spec routed through HelixAgent's own governance.
- **N3.** Mobile/Aurora/Harmony UI work beyond what HelixCode already has, unless a CLI-agent feature port directly requires it.
- **N4.** Migration to a new test framework. We extend the existing testify-based stack.
- **N5.** CI/CD pipelines — Constitution Rule 1 forbids them; we run gates manually or via Makefile.

1.4 Success criteria (the bar for declaring this programme complete)

- **S1.** `git submodule foreach --recursive 'echo OK'` succeeds across the entire HelixCode tree.
 - **S2.** `make ci-validate-all` (root) and `make test-full` (inner) both pass with `zero t.Skip()` that lacks a `SKIP-OK: #<ticket>` marker.
 - **S3.** Every CLI-agent feature listed in `docs/improvements/04/.../porting_*.md` has: (a) source code merged at the documented HelixCode path, (b) a Challenge script that exercises the feature end-to-end against real infrastructure, (c) the Challenge's PASS evidence pasted in its commit message.
 - **S4.** Every governance triplet (Constitution.md / CLAUDE.md / AGENTS.md) at every owned-by-us repo carries the three constitutional anchors: anti-bluff (Article XI §11.9), no-force-push (CONST-043), no-secret-leak (CONST-042).
 - **S5.** SonarQube and Snyk both run locally via `make scan-sonarqube` / `make scan-snyk`, produce real reports, and have at least one finding triaged or waived with a documented exception.
 - **S6.** Refreshed PNG diagrams in `docs/improvements/06_diagrams_real/` reflect the real submodule topology (not the aspirational HelixML/HelixSDK fiction).
-

2. Topology + repo wiring

2.1 Final repo layout (post-Phase-0)

```
helix_code/                                     # meta-repo (this repo)
├── helix_code/                                # tracked subdirectory (Go app, N
│   ├── CLAUDE.md / AGENTS.md / CONSTITUTION.md # NEW — governance tripl
│   ├── cmd/ internal/ applications/ tests/ ...
│   └── go.mod                                 # module dev.helix.code, go 1.26
└── helix_agent/                              # NEW SUBMODULE — git@github.com:
    ├── HelixLLM/                             # nested submodule (canonical)
    ├── HelixMemory/                         # nested submodule (canonical)
    ├── HelixSpecifier/                     # nested submodule (canonical)
    ├── LLMsVerifier/                       # nested submodule (overlaps submo
    ├── cli_agents/                          # 39 CLI-agent submodules — canoni
        ├── claude-code/ aider/ cline/ codex/ continue/ ...
```

```

├── helix_qa/                                # existing submodule
├── challenges/                             # existing submodule (now with con
├── containers/                             # existing submodule
├── security/                               # existing submodule

├── dependencies/HelixDevelopment/          # keep as-is for direct Go imports
│   ├── LLMsVerifier/                       # canonical pin (helix_agent/LLMsV
│   └── DocProcessor/ LLMOrchestrator/ LLMProvider/ VisionEngine/

├── Example_Projects/                       # DEPRECATE — replaced by helix_ag
│   └──                               # (preserved through Phase 4; Phas

├── assets/ github_pages_website/ dependencies/{HuggingFace_Hub,LLama_CP
└── docs/ scripts/ Makefile setup.sh helix CLAUDE.md ...

```

2.2 Submodule rules

- **Protocol: SSH only.** `git@github.com:...` or `git@gitlab.com:...`. Constitution Rule 3 already prohibits HTTPS — re-affirmed.
- **Recursion: deep.** `git submodule update --init --recursive --jobs 8`. `setup.sh` wraps this; we add a verifier that fails if any submodule is uninitialised.
- `helix_code/helix_code/` **stays a tracked subdirectory**. Promoting it to a submodule would create a circular reference (this repo *is* `HelixDevelopment/` `HelixCode`). Documented explicitly in `helix_code/helix_code/CLAUDE.md`.
- **LLMsVerifier dual-pinning.** `submodules/llms_verifier` is the canonical pin used by Go imports; `helix_agent/LLMsVerifier` is `HelixAgent`'s transitive view. `scripts/verify-llmsverifier-pin-parity.sh` fails if pointers diverge.
- **Agent-Deck nested-worktree fix.** `Example_Projects/Agent-Deck/.claude/worktrees/agent-*` paths are git worktrees, not submodules. Add to `.git/info/exclude` (local) and document.

2.3 Secret handling

CONST-042 (NEW): > No API key, token, password, certificate, or other credential may be committed to any repository owned by `HelixDevelopment` or `vasic-digital`, transitively or otherwise. All secrets live in `.env` files (mode 0600) listed in `.gitignore`. Any leak — to git, logs, build artefacts, screenshots, or external services — is a release blocker until rotated and post-mortemed.

Implementation: - `helix_code/helix_code/.env` — copied (not symlinked) from `../helix_agent/.env` during Phase 0; mode 0600; owner-only. - `helix_code/helix_code/.env.example` — every key from real `.env` with `<REDACTED>` placeholders; under git. - `.gitignore` (`root+helix_code/helix_code/`) — explicit `.env`, `.env.local`, `.env.*` (with `!.env.example` exception), `*.pem`, `*.key`, `*.crt`, `id_rsa*`, plus `helix.security.json` if it ever holds secrets. - **Secret-scan gate** — `scripts/scan-secrets.sh` runs `gitleaks` (or fallback `grep` over working tree); wired into `make ci-validate-all`. Failing it blocks any phase work. - **Migration steps (Phase 0 P0-04):** `cp -p ../helix_agent/.env helix_code/helix_code/.env` `chmod 600 helix_code/helix_code/.env` `ls -la helix_code/helix_code/.env`
must show `-rw-----` `git check-ignore helix_code/`
`helix_code/.env` # must exit 0

2.4 Push protections

CONST-043 (NEW): > No force push, force-with-lease push, history rewrite, branch deletion of `main/master`, or upstream-overwriting operation may be performed without explicit, in-conversation user approval given for that specific operation. Authorization for one push does not extend to subsequent pushes. Bypassing hooks (`--no-verify`), signature verification (`--no-gpg-sign`), or protected-branch rules also requires explicit approval. This applies to every repository in the HelixDevelopment / vasic-digital stack.

Implementation: - Pre-push hook at `scripts/git_hooks/pre-push` rejects `--force / --force-with-lease` unless `HELIX_FORCE_PUSH_APPROVED=1` is set. Idempotent installer at `scripts/install-git-hooks.sh` invoked from `setup.sh`. - Hook is local courtesy gate; the constitutional clause is the actual contract.

2.5 Authorisation scope for non-force pushes during this programme

The user has authorised, in this conversation, **non-force pushes of new commits to existing branches on the four configured remotes** for the duration of this programme — without per-push approval. Force pushes, new branches, new remotes, hook bypasses: still per-operation approval.

3. Phase 0 — Foundation Cleanup

Phase 0 is **the gate**. Nothing in Phases 1-5 begins until P0 is verified done. Every task has an acceptance check; the phase closes only when every check passes and a single rolled-up evidence log is committed at `docs/improvements/05_phase_0_evidence.md`.

3.1 Phase 0 task list

#	Task	Acceptance check
P0-01	Resolve <code>Example_Projects/Agent-Deck/.claude/worktrees/</code> recursion error: add path to <code>.git/info/exclude</code> (local); document fix in <code>helix_agent/cli_agents/agent-deck/CLAUDE.md</code> (after P0-02)	<code>git submodule</code> returns ≥ 87 with no
P0-02	Add HelixAgent submodule: <code>git submodule add git@github.com:HelixDevelopment/HelixAgent.git</code> <code>HelixAgent && git submodule update --init --recursive HelixAgent</code>	<code>ls helix_agent</code> {HelixLLM,HelixClaude-code} and
P0-03	<code>scripts/verify-llmsverifier-pin-parity.sh</code> — fails if <code>submodules/llms_verifier</code> SHA differs from <code>helix_agent/LLMsVerifier</code> SHA	Script exits 0 when <code>validate-all</code>
P0-04	Migrate API keys: <code>cp -p ../helix_agent/.env helix_code/helix_code/.env && chmod 600 helix_code/helix_code/.env</code>	<code>ls -la</code> shows <code>-rw</code>
P0-05	Update <code>.gitignore</code> (root + <code>helix_code/helix_code/</code>): <code>.env</code> , <code>.env.local</code> , <code>.env.*</code> with <code>!.env.example</code> , plus <code>*.pem *.key *.crt id_rsa*</code>	<code>git status --</code> <code>grep -E '\.env'</code>
P0-06	Refresh <code>helix_code/helix_code/.env.example</code> : every key from <code>../helix_agent/.env</code> with placeholder values; no real values	

#	Task	Acceptance check
		<code>diff <(grep -oE '^[A-Z_]+=</code> empty
P0-07	<code>scripts/scan-secrets.sh</code> — gitleaks or fallback grep for <code>sk-</code> , <code>gho_</code> , <code>glpat-</code> , <code>xoxb-</code> , <code>AKIA</code> , <code>eyJ</code> ; wired into <code>make ci-validate-all</code>	Exits 0 on clean tree
P0-08	<code>scripts/git_hooks/pre-push</code> rejecting <code>--force</code> unless <code>HELIX_FORCE_PUSH_APPROVED=1</code> ; idempotent installer at <code>scripts/install-git-hooks.sh</code> invoked from <code>setup.sh</code>	Hook blocks <code>git push</code> only — do not actually
P0-09	Create governance triplet for <code>helix_code/helix_code/</code> : <code>CONSTITUTION.md</code> , <code>CLAUDE.md</code> , <code>AGENTS.md</code> , each derived from root + Go-module-specific addenda	All three files exist;
P0-10	Add <code>CONST-042</code> + <code>CONST-043</code> to root <code>CONSTITUTION.md</code> Article XII (NEW)	<code>grep -E "CONST</code>
P0-11	Cascade <code>CONST-042</code> + <code>CONST-043</code> to root <code>CLAUDE.md</code> , <code>AGENTS.md</code> , <code>CRUSH.md</code> , <code>QWEN.md</code> . Backfill anti-bluff anchor to <code>CRUSH.md</code> and <code>QWEN.md</code> (currently missing)	<code>for f in CLAUDE</code> <code>"11.9\ CONST-0</code>
P0-12	Cascade all three anchors to every owned submodule: <code>HelixQA</code> , <code>Challenges</code> , <code>Containers</code> , <code>Security</code> , <code>dependencies/HelixDevelopment/{LLMsVerifier,DocProcessor,LLMOrchestrator,LLMProvider,VisionEngine}</code> , plus new <code>HelixAgent</code> and its nested <code>HelixLLM/HelixMemory/HelixSpecifier</code> . Backfill missing anti-bluff anchor in <code>LLMsVerifier</code> (all 3) and <code>CONSTITUTION.md</code> anchor in the four <code>Dependencies</code> repos	<code>scripts/verify</code>
P0-13	Fix root <code>CLAUDE.md</code> §3.2 bluff: change <code>helix_code/ ← SUBMODULE</code> to <code>helix_code/ ← TRACKED SUBDIRECTORY</code> (NOT a submodule — meta-repo's primary inner directory; circular reference if promoted)	<code>grep -A1 '^ </code>
P0-14	Wire P0 gates into Makefile: <code>make verify-foundation</code> runs P0-01, 03, 05, 07, 12 checks plus <code>no-silent-skips</code> and <code>verify-governance-cascade.sh</code>	Exits 0 with no warn
P0-15	Refresh PNG diagrams: regenerate <code>overall_architecture.png</code> , <code>dependency_graph.png</code> , <code>feature_gap_matrix.png</code> , <code>integration_phases.png</code> against real module set. Output to <code>docs/improvements/06_diagrams_real/</code> ; deprecate 01/02 with <code>DEPRECATED.md</code> pointers	New diagrams exist; location
P0-16	Write <code>docs/improvements/05_phase_0_evidence.md</code> — pasted output of every P0 acceptance check with timestamps. Single atomic commit listing all 16 tasks. Push to all four remotes (no force)	<code>git ls-remote</code> shows matching SH

3.2 Phase 0 estimated effort

~1-2 working sessions. Mostly mechanical (governance cascade is templated). Tricky parts: P0-04 (secret migration), P0-08 (hook semantics), P0-15 (diagram regen).

4. Phases 1-5 charters

Each phase is its own future spec → plan → implementation cycle. The synthesis spec only commits to **scope, dependencies, acceptance criteria** — per-feature implementation detail goes into that phase’s own writing-plans cycle.

4.1 Phase 1 — claude-code-source porting (priority #1)

Scope (frozen): All 20 features in docs/improvements/04/.../porting_claude_code.md: 1. Auto-Compaction System 2. Permission Rule System 3. Tool Result Persistence 4. Git Worktree Agent Isolation 5. Hook-Based Extensibility 6. MCP Full Lifecycle 7. Background Task System 8. Plan Mode 9. Slash Command System 10. Skill System 11. Session Transcript Resume 12. Multi-Provider Backend 13. LSP Integration 14. Sandboxed Shell Execution 15. Subagent Team 16. OpenTelemetry Integration 17. Smart File Editing 18. No-Flicker Rendering 19. AskUserQuestion with Previews 20. Theme System

Source of truth: helix_agent/cli_agents/claude-code/ (canonical) — *not* helix_code/Example_Projects/Claude_Code/. claude-code core is closed-source; porting derives features from public SDK + observable CLI behaviour. No reverse-engineering of binaries.

Per-feature deliverables (all six required): - (a) Source landed at the documented HelixCode path under helix_code/internal/<package>/. - (b) Unit tests using existing testify pattern; mocks allowed only here. - (c) Integration test under helix_code/tests/integration/ with -tags=integration, **no mocks**, real PostgreSQL/Redis. - (d) Challenge script under helix_code/tests/e2e/challenges/<feature>/ against the docker-compose-full-test stack with positive runtime evidence. - (e) Updated reference docs in docs/COMPLETE_API_REFERENCE.md / docs/COMPLETE_CLI_REFERENCE.md if applicable. - (f) Anti-bluff smoke check — `grep -rn "simulated\|for now\|TODO implement\|placeholder" <new_files>` empty.

Acceptance criteria for Phase 1: - All 20 features land with full (a)-(f). - `make test-full` (with `make test-infra-up`) passes with zero new `t.Skip()` without `SKIP-OK` markers. - Each feature’s Challenge has runtime evidence pasted in its commit message body. - Features are wired into applications/desktop/, applications/terminal_ui/, cmd/cli/ where applicable.

Estimated cycle: 6-10 working weeks.

4.2 Phase 2 — Remaining CLI agents porting

Priority order:

Order	Agent	Source	Highlight features
1	Aider	helix_agent/cli_agents/aider/	Architect/Editor dual-model, 4 matching, repo-map (tree-sitter), diff, voice-to-code, IDE watch, prompt caching
2	Cline	helix_agent/cli_agents/cline/	Plan/Act modes, browser auto watching, MCP marketplace, task chains

Order	Agent	Source	Highlight features
3	OpenHands	helix_agent/cli_agents/openhands/	Sandbox runtimes (Docker/E2E), agent micro-framework, evaluation harness, multi-agent delegation
4	Codex	helix_agent/cli_agents/codex/	Approval workflows, sandbox, multi-turn refinement
5	Continue	helix_agent/cli_agents/continue/	IDE-native context, slash commands, custom providers
6	Forge	helix_agent/cli_agents/forge/	Multi-tier model routing, retrieval, optimisation
7	Plandex	helix_agent/cli_agents/plandex/	Long-running plans with branching, ceiling, partial-apply
8	Kilo Code	helix_agent/cli_agents/kilo-code/	Mode profiles, custom modes, integration
9	GPT-Engineer	helix_agent/cli_agents/gpt-engineer/	Project bootstrapping from prompts, by-file generation
10	OpenCode (sst)	helix_agent/cli_agents/opencode/	TUI patterns, session management, switching
11	Gemini-CLI	helix_agent/cli_agents/gemini-cli/	Long-context strategies, Gemini tool calling
12	DeepSeek-CLI	helix_agent/cli_agents/deepseek-cli/	Reasoning-mode wrapper, prompts
13	GPTme	helix_agent/cli_agents/gptme/	Local-first agent, terminal-native, plugin ecosystem
14	Qwen-Code	helix_agent/cli_agents/qwen-code/	Qwen-tuned reasoning prompts, specific tool calling
15	TaskWeaver	helix_agent/cli_agents/taskweaver/	Code-as-action orchestration, planner-executor
16	Codex (open-source CLI variant)	helix_agent/cli_agents/codex/	Approval workflows, sandbox, multi-turn refinement (clarify whether this points at OpenAI source CLI or the legacy closed-source the latter, move to N1)
17	Bundled remainder	helix_agent/cli_agents/{octogen,multiagent-coding,nanocoder,ollama-code,get-shit-done,vtcode,conduit,bridle,noi,roo-code,spec-kit,swe-agent,junie,agent-deck,codename-goose,fauxpilot,gpt-engineer-sub-variants,...}	Single sub-spec (porting_remaining_agents); per-agent feature audit done at build time; agents lacking unique features noted as “no porting needed, a reference”
18	MCP/skill supplements	helix_agent/cli_agents/{git-mcp,postgres-mcp,claude-	Not agents — MCP servers and packs that supplement Phase 1 (MCP Lifecycle, Skill System)

Order	Agent	Source	Highlight features
		plugins, codex-skills, ui-ux-pro-max}/	during the corresponding Phase work

Phase 2 scope is exhaustive: every subdirectory under `helix_agent/cli_agents/` is in scope unless it appears in the N1 closed-source list. Agents not enumerated above by name are covered by row 17 (Bundled remainder). At Phase 2 sub-spec time, do `ls helix_agent/cli_agents/` and reconcile every entry against this table; any new agent that has appeared upstream since spec authoring gets added to row 17 and processed.

Out of scope per N1 (closed-source / no portable code): `claude-squad`, `amazon-q`, `copilot-cli`, `warp`, `kiro-cli`, `shai`. Treat as observable behaviour studies if UX patterns inspire features. `superset`, `snow-cli` — note that despite being in `cli_agents/`, these are non-agent products (Apache Superset analytics, Snowflake CLI) and contribute no porting features; document and skip.

Per-agent deliverable shape: identical to Phase 1 (a)-(f).

Acceptance criteria for Phase 2: - Every priority-listed agent has its sub-spec executed; features land under `helix_code/internal/<package>/` or `helix_code/applications/...` - No silent feature skips — intentional omissions get SKIP-OK: `<reason>` in the porting doc. - Fusion goal verified: a single end-user can invoke `claude-code`'s Plan Mode on a task using Aider's repo-map + Cline's browser automation + OpenHands' sandbox — and it works end-to-end.

Estimated cycle: 8-12 working weeks.

4.3 Phase 3 — Test infrastructure expansion (parallelisable with Phase 2)

Test-type matrix:

Type	Mocks?	Folder	Acceptance
Unit	✓ allowed	<code>tests/unit/</code> + per-package <code>*_test.go</code>	100% line coverage every internal <code><pkg>/</code> (new sufficient)
Integration	×	<code>tests/integration/</code>	- <code>tags=integration</code> real PostgreSQL Ollama; every path
E2E	×	<code>tests/e2e/challenges/<feature>/</code>	Per-feature Coverage with runtime
Full-automation	×	<code>tests/automation/</code>	Headless multi workflows
Performance	×	<code>tests/performance/</code>	Latency P50 budgets per path endpoint
Concurrency	×	<code>tests/concurrency/</code> (NEW)	- race enforcement property tests mutex paths

Type	Mocks?	Folder	Acceptance
Benchmarking	×	tests/performance/benchmarks/	go test - baseline per regression re
Security	×	tests/security/	SQL-injection bypass, tenar JWT edge ca
Scanning	n/a	scripts/scan- {sonarqube,snyk,gitleaks,trivy,grype,osv}.sh	All scanners SonarQube + mandatory
DDoS	×	tests/security/ddos/ (NEW)	Rate-limit ve with vegeta c HTTP 429 v
Challenge banks	×	challenges/banks/<bank-name>/	Each bank ca runtime evid
helix_qa banks	×	helix_qa/banks/<bank-name>/	Heavy QA se multi-platfor verification

Acceptance criteria for Phase 3: - make test-full runs all test types; zero t.Skip() without justified markers. - scripts/scan-* all run via make scan-all; reports archived under docs/scan-reports/<date>/. - Concurrency suite passes with -race. - DDoS suite confirms rate limiting; thresholds documented.

Dependencies: Phase 0 done. Parallelisable with Phases 1-2 once scaffolding is up.

4.4 Phase 4 — Anti-bluff verification pass

Activities: - Run bluff-detector.sh over the entire tree → BLUFF_AUDIT_FINDINGS.md listing each finding (file, line, class, severity). - Triage each: **fix** (tighten test/Challenge), **waive** (with SKIP-OK: #<ticket> and written justification), or **delete** (test was meaningless from inception). - Re-run; commit tightened tests + waiver list. - Sample 10% randomly; user (or fresh terminal session on clean machine) re-executes; evidence must match. - Output: BLUFF_AUDIT_REPORT.md under docs/improvements/.

Acceptance criteria for Phase 4: - Bluff-detector exits clean. - Every previously-skipped test removed (with rationale) or carries SKIP-OK: #<ticket> tied to real defect. - 10% random re-run sample produces matching evidence.

Dependencies: Phases 1+2 done; Phase 3 infrastructure ready.

4.5 Phase 5 — End-user materials uplift

Deliverables: - helix_code/helix_code/docs/USER_MANUAL.md — full rewrite covering every Phase-1+2 feature; rendered to HTML at github_pages_website/manual/. - github_pages_website/ — feature pages, comparison-with-{claude-code,aider,cline} matrix, screenshots/asciinema demos. - docs/VIDEO_COURSE_CURRICULUM.md — per-feature episode outlines with sample scripts (full filming out of scope). - docs/adr/NNN-<topic>.md — one ADR per non-trivial port. -

docs/improvements/06_diagrams_real/ — refreshed showing all ported features. - Per-owned-submodule README — call out role in HelixCode integration.

Acceptance criteria for Phase 5: - A new user can follow USER_MANUAL.md from install to running a multi-agent task without external help. - github_pages_website/ builds and renders. - Every Phase-1/2 feature has at least one mention in user-facing docs.

Dependencies: Phases 1+2+4 done.

4.6 Cross-phase rules

- **No phase declares done** without a single rolled-up evidence file (docs/improvements/0N_phase_N_evidence.md) committed and pushed (no force) to all four remotes.
- **Every phase respects CONST-035, CONST-042, CONST-043.**
- **Submodule pointer updates** require running through secret-scan + verify-pin-parity gate.
- **Force-push prohibition is absolute.**

5. Anti-bluff verification framework

5.1 Bluff taxonomy

#	Class	Symptom	Example
B1	Metadata-only PASS	Test asserts a struct field exists, never checks the field's value matches expected behaviour	<code>require.NotNil(t, user.Token)</code> without verifying token is JWT-shaped or accepted by an auth server
B2	Configuration-only PASS	Test verifies config loads, never verifies the configured component does anything	<code>cfg, err := config.Load(...); require.NoError(t, err)</code> and stops
B3	Absence-of-error PASS	Test calls a function expecting <code>nil</code> error, never inspects what was produced	<code>_, err := provider.Generate(ctx, req); require.NoError(t, err)</code> with no follow-up assertion
B4	Grep-based PASS	Challenge greps source for a string, calls that “verification”	<code>grep -q "os.Exec" cmd.go && echo PASS</code>
B5	Mock-only integration	Test labelled “integration” but uses <code>mockDB.On(...).Return(...)</code>	Forbidden by Constitution Rule 5 + Rule 2; appears in some <code>tests/integration/subtrees</code>
B6	Print-and-sleep simulation	Code “executes” by printing the command then <code>time.Sleep</code> instead of <code>os/exec</code>	The original BLUFF-003
B7	Skipped-feature PASS	Test starts with <code>if testing.Short()</code>	Common in CI-shy codebases

#	Class	Symptom	Example
		{ t.Skip() } and is always run with -short	

5.2 Detection — scripts/bluff-detector.sh

A composite scanner; wired into make ci-validate-all. Components:

- **Check 1 — t.Skip() audit (B7):** every t.Skip(), testing.Short(), build-tag exclusion must have SKIP-OK: #<ticket> within 3 lines OR be flagged. Existing scripts/no-silent-skips.sh reused + extended.
- **Check 2 — assertion-density audit (B1, B2, B3):** for every *_test.go, count NoError/assert.NoError ratio vs. behavioural assertions (assert.Equal, assert.Contains, assert.JSONEq, custom matchers). NoError-density > 60% AND behavioural-assertion count < 2 → flagged for human review (smell, doesn't gate).
- **Check 3 — challenge runtime-evidence audit (B4):** every tests/e2e/challenges/<feature>/run.sh must (1) make a real network call (curl/nc/psql/redis-cli/docker exec) OR (2) spawn a real subprocess via go run / ./bin/helixcode AND (3) print at least one line containing the feature's expected output pattern. Greps without (1) or (2) → fail.
- **Check 4 — integration-test purity (B5):** files matching tests/integration/**/*_go with -tags=integration build constraint must NOT import internal/mocks/. grep -rn "internal/mocks" helix_code/tests/integration/ empty.
- **Check 5 — simulation-string scan (B6):** grep -rn "simulated\\|for now\\|TODO implement\\|placeholder\\|This is a simulated\\|simulate generation\\|For now," helix_code/internal helix_code/cmd helix_code/applications empty.
- **Check 6 — print-and-sleep pattern:** AST scan flags any function with fmt.Print* followed by time.Sleep followed by return nil with no exec.*Command between them.

5.3 Runtime evidence — what counts

Every Challenge has a sibling expected.json:

```
{
  "must_appear_in_stdout": ["regex1", "regex2"],
  "must_not_appear": ["simulated", "placeholder"],
  "min_duration_ms": 100,
  "max_duration_ms": 30000,
  "asserts_external_state": [
    {"type": "postgres", "table": "users", "rowcount_min": 1},
    {"type": "redis", "key_pattern": "session:*", "min_keys": 1},
    {"type": "http", "url": "http://localhost:8080/api/v1/health", "expect_status": 200}
  ]
}
```

The Challenge runner (`tests/e2e/challenges/cmd/runner/main.go`) parses and asserts. **A Challenge cannot self-certify** — it does not write its own PASS verdict; the runner does, after evaluating evidence.

5.4 Retroactive bluff audit (Phase 4)

1. Run `bluff-detector.sh` over the whole tree → `BLUFF_AUDIT_FINDINGS.md`.
2. Triage each finding: fix / waive / delete.
3. Re-run; commit tightened tests + waiver list.
4. Sample 10% randomly; user (or fresh session on clean machine) re-executes; evidence must match.
5. Output `BLUFF_AUDIT_REPORT.md` under `docs/improvements/`.

5.5 CONST-035 enforcement loop (per ported feature)

For every feature port: 1. Write production code. 2. Write unit test (mocks allowed). 3. Write integration test (no mocks). 4. Write Challenge with `expected.json`. 5. Run Challenge against `make test-infra-up`. 6. Paste runner output (with timestamps) into commit message body. 7. Run `bluff-detector.sh` on diff; must exit clean. 8. Only then is the feature “done.”

6. Documentation cascade

6.1 Constitution structure

Every owned-by-us repo’s governance triplet (`CONSTITUTION.md`, `CLAUDE.md`, `AGENTS.md`) carries the **same three constitutional anchors**, with the **same wording**, in the **same article numbering**. Wording lives in canonical templates at the meta-repo root; `scripts/propagate-governance.sh` rewrites the corresponding section in each submodule on every run.

Article XI §11.9 — Anti-Bluff (existing, expanding cascade): > Verbatim user mandate: *“We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can’t be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!”* > Operative rule: every PASS carries positive runtime evidence captured during execution. Metadata-only / configuration-only / absence-of-error / grep-based PASS without runtime evidence are critical defects.

Article XII (NEW) — Repository Safety: - §12.1 (CONST-042) — No-Secret-Leak. No API key, token, password, certificate, or other credential may be committed to any repository owned by HelixDevelopment or vasic-digital. All secrets in `.env` (mode 0600) listed in `.gitignore`. Any leak is a release blocker until rotated and post-mortemed. - **§12.2 (CONST-043) — No-Force-Push.** No force push, force-with-lease push, history rewrite, branch deletion of `main/master`, or upstream-overwriting operation without explicit per-operation user approval. Authorization for one push does not extend further.

6.2 CLAUDE.md / AGENTS.md structure

Both files contain: - **§ Peer Governance** — pointer to sister files, links to parent `helix_code/`. - **§ Mandatory Rules** — full enumeration of every applicable CONST-xxx with one-line summary. - **§ Anti-Bluff** — Article XI §11.9 verbatim quote + operative rule +

cascade requirement. - § **Repo-Specific Addenda** — only here may submodules diverge. Tech stack, build commands, single-test invocation. Delimited by `<!-- BEGIN: REPO-SPECIFIC ADDENDA -->` / `<!-- END: REPO-SPECIFIC ADDENDA -->` markers. - § **Reference Commands** — concrete invocations: build, test, scan, verify-foundation.

`scripts/propagate-governance.sh` enforces the first three sections by replacement; addenda preserved.

6.3 Cascade automation

```
canonical/CONSTITUTION_ANCHORS.md      # canonical wording for the three a
canonical/CLAUDE_TEMPLATE.md            # template with addenda placeholder
canonical/AGENTS_TEMPLATE.md
owned-repos.txt                          # HelixCode (root), helix_code/Heli
                                         # HelixAgent, helix_agent/HelixLLM,
                                         # helix_agent/HelixSpecifier, Helix
                                         # Containers, Security, dependencie
                                         # NOT included: cli_agents/* (third
                                         #                               dependencies/{0lla
```

Run via `make propagate-governance`. Verifier `scripts/verify-governance-cascade.sh` (extended) checks all three anchors in every owned-by-us governance file; failing it blocks `make ci-validate-all`.

6.4 End-user materials

Inside `helix_code/helix_code/`: - `docs/USER_MANUAL.md` — end-to-end user guide (Phase 5); HTML via `make manual-html`. - `docs/COMPLETE_API_REFERENCE.md`, `docs/COMPLETE_CLI_REFERENCE.md`, `docs/COMPLETE_CONFIGURATION_DOCUMENTATION.md`, `docs/COMPLETE_DEPLOYMENT_GUIDE.md`, `docs/COMPLETE_TROUBLESHOOTING_GUIDE.md` — refreshed per port. - `docs/adr/` — ADRs added per non-trivial port. - `docs/HOST_POWER_MANAGEMENT.md` — already documents CONST-033; cross-link to CONST-042/042.

Inside meta-repo root: - `github_pages_website/` — feature pages, comparison matrix, install guide, asciinema demos. - `Documentation/` — mirrored canonical docs. - `website/` — marketing site. - `docs/improvements/06_diagrams_real/` — refreshed PNGs (P0-15). - `docs/improvements/05_phase_0_evidence.md` — P0 rolled-up evidence. - `docs/improvements/0N_phase_N_evidence.md` — one per phase. - `docs/improvements/01_*/DEPRECATED.md`, `02_*/DEPRECATED.md` — pointers to `06_diagrams_real/`.

Per owned submodule: `README.md` updated with “Role in HelixCode” section.

6.5 Video curriculum (Phase 5, scope-limited)

`docs/VIDEO_COURSE_CURRICULUM.md`: - Curriculum outline (modules + episodes). - Per episode: title, learning objectives, ~3-5 minute outline, sample script intro+outro, screen-recording cue list. - **Out of scope**: filming/editing.

6.6 Diagram regeneration

`scripts/regenerate-diagrams.py` (NEW; replaces 01/02 generators) reads `docs/improvements/canonical/topology.yaml` (real module set) and emits to `docs/improvements/06_diagrams_real/`: - `overall_architecture.png` — hub-and-spoke with HelixCode core, HelixAgent substrate, real Helix-* libraries. - `dependency_graph.png` — topological layers from actual `go.mod + .gitmodules`. - `feature_gap_matrix.png` — matrix against real modules; cells based on actual feature presence. - `integration_phases.png` — refreshed timeline.

Old 01/02 PNGs preserved as historical artefacts; each gets `DEPRECATED.md` pointer.

6.7 Cascade enforcement gate

`make verify-foundation` (P0 deliverable, extended through later phases) runs: 1. `scripts/verify-governance-cascade.sh` — all anchors present in every owned-by-us repo. 2. `scripts/scan-secrets.sh` — no leaked credentials. 3. `scripts/verify-llmsverifier-pin-parity.sh` — LLMsVerifier dual-pin parity. 4. `scripts/no-silent-skips.sh` — every `t.Skip()` justified. 5. `scripts/bluff-detector.sh` — no detected bluff patterns.

Runs before any phase declares itself done.

7. Operational continuity

7.1 Single source of truth: `docs/improvements/PROGRESS.md`

A markdown file at a stable path that any future session — including a fresh agent with no conversation history — can read to understand exactly where the programme stands.

HelixCode CLI-Agent Fusion — Live Progress Tracker

```
> **STOP/RESUME PROTOCOL**: read this file first. The "current
    focus" pointer
> below identifies the active task. The "evidence trail" links
    every claim of
> "done" to its commit + Challenge output.
```

Current focus

```
- **Active phase:** P<n> — <phase name>
- **Active task:** P<n>-<id> — <task subject>
- **Owner:** <user-or-agent>
- **Started:** <ISO timestamp>
- **Last touched:** <ISO timestamp>
- **Blocked-on:** <none | ticket | upstream issue>
```

Phase status

```
| Phase | State | Started | Completed | Evidence |
| --- | --- | --- | --- | --- |
```

```
| P0 — Foundation | <pending/active/done> | — | — | docs/
|           improvements/05_phase_0_evidence.md |
| P1 — claude-code | ... | ... | ... | ... |
| P2 — Other CLI agents | ... | ... | ... | ... |
| P3 — Test infra | ... | ... | ... | ... |
| P4 — Anti-bluff audit | ... | ... | ... | ... |
| P5 — End-user materials | ... | ... | ... | ... |
```

Active phase task list

```
- [x] P<n>-01 — <subject> ← commit <sha>
- [-] P<n>-02 — <subject> ← in progress
- [ ] P<n>-03 — <subject>
```

Decision log

```
- <date> — <decision> — rationale — link to commit/discussion
```

Open risks / parking lot

```
- <item>
```

Updated atomically with every commit that advances the programme.

7.2 Commit cadence

Every meaningful unit of progress = one commit. A “meaningful unit” is at minimum: - One Phase task moved from `in_progress` → `completed`, OR - One Phase task’s intermediate sub-step that lands code or docs.

No squashing during the programme. Each commit message body includes:

```
<type>(<phase-task>): <subject>
```

```
<short description>
```

```
Phase: P<n>
```

```
Task: P<n>-<id>
```

```
Evidence: <pasted runtime output OR pointer to file containing it>
```

```
Co-Authored-By: Claude Opus 4.7 (1M context) <noreply@anthropic.com>
```

7.3 Push cadence

Default cadence: push at the end of every working session, and at every phase boundary.
Working session = continuous block of agent-driven work (typically 1-3 hours of clock time).

Push protocol (cascade order — submodule first, then parent): 1. For every submodule with new commits in the session: 1. `cd <submodule> && make verify-foundation` (when the submodule has it; otherwise compile check) 2. `cd <submodule> && scripts/scan-secrets.sh` (if available; else fallback `grep`) 3. `cd <submodule> && git push <each-remote> <branch>` — every configured remote, sequentially 2. For the meta-repo (after all submodules pushed): 1. `make verify-foundation` 2. `scripts/scan-secrets.sh` 3. `git add updated submodule pointers + PROGRESS.md + meta-repo files` 4. Single commit listing every submodule SHA bumped 5. `git push <each-remote> main`

for github, gitlab, origin, upstream 3. Verify: for r in github gitlab origin upstream; do git ls-remote --heads \$r main; done — all four return same SHA.

Authorisation rule (CONST-043 reaffirmed): - Regular git push (no force) for advancing main on existing remotes during this programme — pre-authorised in conversation; no per-push approval needed. - Force pushes / history rewrites / hook bypasses / new branches / new remotes — still per-operation approval.

Pre-push gate (mechanical):

```
make verify-foundation && scripts/scan-secrets.sh && \
  for r in github gitlab origin upstream; do git push $r <branch>; done
```

If verify-foundation or secret scan fails → push aborts; evidence captured in PROGRESS.md “blocked-on” field.

7.4 Recovery semantics

Any agent / human starting a fresh session executes, in order: 1. git -C <repo> submodule update --init --recursive --jobs 8 2. cat docs/improvements/PROGRESS.md — get current focus. 3. git log --oneline -10 docs/improvements/PROGRESS.md — verify file is current. 4. make verify-foundation — confirm foundation intact. 5. Resume the active task per the file.

If PROGRESS.md is stale or contradicts git state, fix it first; never start work on top of inconsistent state.

7.5 Stop semantics

User says “stop”, session times out, or work pauses for any reason: 1. Finish the in-flight tool call cleanly. 2. Update PROGRESS.md: move active task to completed (if cleanly done) or back to pending with stopped at: <description> note. 3. Commit + push per cadence above. 4. Report stop status to user.

At no point is the system left with in-flight work that isn’t either completed-and-committed or rolled-back-to-clean-restartable state.

8. Spec self-review log

Pass executed 2026-05-04 by the authoring agent immediately before handing the spec to user review.

Findings & resolutions: 1. **Placeholder scan** — flagged hits at the bluff-detector grep patterns (TODO implement, placeholder strings) and inside the PROGRESS.md template (<n>, <id>, <subject>). All confirmed as legitimate content (literal strings used by detector, runtime template tokens). **No fix needed.** 2. **Internal consistency** — Phase boundaries, anchor names (CONST-035 / -041 / -042), and acceptance-criteria references match across §1, §3, §4, §5, §6. CONST numbering matches the existing root CONSTITUTION.md style. **No fix needed.** 3. **Scope check** — single spec covers a multi-phase programme but each phase has its own writing-plans cycle; this spec only commits to scope, dependencies, acceptance criteria, and operational continuity. The decomposition is appropriate for the brainstorming → writing-plans

handoff. **No fix needed.** 4. **Ambiguity check** — push authorisation rule clarified: regular non-force pushes pre-authorised for the duration of the programme; force pushes / hook bypasses / new branches / new remotes still require per-operation approval. **§7.3 was already explicit; reaffirmed during review.** 5. **Coverage of “ALL CLI agents” mandate** — original Phase-2 priority table only enumerated 13 explicit agents + a “bundled remainder” without specifying the bundled set. The user’s mandate is “all CLI agents” with no exceptions. **FIXED inline:** Phase 2 table extended to row 18; explicit clause added that scope is every `helix_agent/` `cli_agents/` subdirectory minus N1 closed-source set, with a Phase-2-time reconciliation step. 6. **Codex ambiguity** — spec was unclear whether “Codex” refers to OpenAI’s legacy closed-source CLI or the modern open-source successor. **FIXED inline:** row 16 explicitly defers the disambiguation to the Phase 2 sub-spec; if it’s the closed variant it moves to N1. 7. **Decision-log column in PROGRESS.md** — present and used for in-band-but-out-of-spec calls (e.g., the Codex disambiguation, any submodule pointer rollback). Adequate.

Open spec-level uncertainty (deferred to writing-plans): - Exact Go module import path under `dev.helix.code/helix_agent/HelixLLM/...` — needs reading `helix_agent/HelixLLM/go.mod` after P0-02 lands. Recorded as an open item; does not block the spec. - Whether `Example_Projects/` is deleted in Phase 5 or kept as read-only mirror — decided after Phase 4 verification per §2.1. - Submodule depth (`--depth=1` shallow vs. full clone) — measured during P0-02; default to full clone unless a submodule exceeds 500 MB.

9. References

- **Constitution:** `CONSTITUTION.md` (Article XI §11.9 = anti-bluff anchor; Article XII §12.1 = CONST-042; §12.2 = CONST-043 — both NEW in P0-10)
 - **Existing planning material:** `docs/improvements/03_main_plan_step_01/deep_dive_submodule_integration/` and `docs/improvements/04_main_plan_step_02/` `kim_agent_helix_cli_integration_blueprint/`
 - **Per-CLI-agent porting docs:** `docs/improvements/04/.../porting_*.md` (16 documents)
 - **Anti-bluff framework reference:** `docs/improvements/04/.../anti_bluff_test_framework.md`
 - **Real module map (gh org):** `HelixDevelopment/` `{HelixAgent, HelixCode, HelixLLM, HelixMemory, HelixSpecifier, HelixQA, Helix...` and `vasic-digital/{LLMsVerifier, Challenges, Containers, ...}`
-

End of synthesis design spec. Hand-off to superpowers: writing-plans follows user review.